LIBYS: Sovereign Reputation via Delegated Trust Graphs

v0.2 (2026)
Vitor Silva Alves
www.github.com/foiovituh/libys-core

Abstract. A purely decentralized version of reputation would allow interactions to be recorded without the need for a central mediator. We propose LIBYS, a protocol where verifiable interaction events between cryptographic identities form a shared graph. Reputation can be derived by observers from hierarchical domains and controlled by temporary capabilities (contextual authority) granted by the user. The protocol creates a reconstruction burden for identity manipulation or abandonment, ensuring that an identity's utility is proportional to its history of valid and verifiable interactions.

1. Introduction

Digital identity management remains dependent on centralized platforms. With the rise of synthetic agents, trust has become harder to establish and verify. We propose an open infrastructure that unifies identity, reputation, and authority. Integrity is guaranteed by the validity of cryptographic signatures and a delegation structure. The protocol is agnostic to the persistence layer; the trust graph can be distributed via P2P networks (like Nostr [1] or IPFS), decentralized ledgers, or immutable databases.

2. Identity

An identity is a cryptographic key pair based on the Ed25519 curve [2]. The public key serves as the global identifier of the identity. Private keys allow event authorship and must remain secret. Public keys MUST be encoded as 32-byte Ed25519 public keys represented as lowercase hexadecimal strings (64 characters). Creation is permissionless. The protocol assumes that new identities start with zero reputation. Since abandoning an identity implies losing accumulated history, the protocol imposes a structural cost on constant profile recreation (analogous to the energy cost in Proof-of-Work systems [3, 4]). The protocol is structurally symmetric: there is no technical distinction between users, synthetic agents, or applications. All are treated equally as valid cryptographic identities.

3. Interaction Event

An *interaction event* is a flat JSON structure representing a signed declaration. To ensure deterministic hashing without complex canonicalization, the event ID is the SHA-256 hash of a serialized array of its core fields.

3.1. Event Schema

```
{
  "id": "string (hex)",
  "pubkey": "string (hex)",      // Issuer's public key
  "created_at": 1710323400,      // Unix timestamp (seconds)
  "type": "string",              // Hierarchical domain (e.g., api.status.incident)
  "subject": "string",           // Optional: Public key or event ID
  "auth_id": "string",           // Optional: ID of the 'system.auth.grant' event
  "content": "string",           // UTF-8 with domain data (escaped JSON or raw text)
  "sig": "string (hex)"          // Ed25519 signature of the event ID
}
```

The ***subject*** field is polymorphic and may reference either an identity (public key) or an event (by ID), depending on the domain semantics.

3.2. Event ID Determinism

To ensure deterministic hashing across different programming languages, all optional fields (***subject*** and ***auth_id***) MUST be treated as empty strings (`""`) if they carry no value. The event ID is computed as the SHA-256 hash of a serialized JSON array:

```
id = SHA256(UTF8(JSON.stringify([
  pubkey,
  created_at,
  type,
  subject,
  auth_id,
  content
])))
```

Implementations MUST use the exact field order shown above. Observers MUST reject events whose ***id*** does not match the computed hash. The signature ***sig*** MUST be verified using Ed25519 with the event ***pubkey*** as the verification key and ***id*** as the signed message.

3.3 Timestamp Validation

To mitigate timestamp spoofing in decentralized environments, observers MUST reject events whose ***created_at*** timestamp is more than 5 minutes in the future relative to the observer's local clock. Older events MAY be accepted.

3.4. Spam Mitigation (Proof-of-Work)

To mitigate Denial of Service (DoS) attacks and spam on P2P transport layers, implementations MAY require the hash generated in the `id` field to contain a prefix of leading zeros (Proof-of-Work [3, 4]). The PoW difficulty acts as a dynamic write-rate regulator, imposing a verifiable computational cost on event creation. Such mechanisms are applied at the transport or propagation layer and are not part of the core protocol; they do not affect event validity, only whether an event is accepted or relayed by a given node.

4. Domain Taxonomy and Authority

The semantic meaning of events is determined by their domain identifier. Domains MUST follow a three-level hierarchical format (`category.context.action`). Each segment is separated by dots and MUST match the `[a-z0-9]+` pattern. Examples:

- `social.forum.post`
- `api.status.incident`
- `finance.loan.settled`
- `market.order.disputed`

4.1. Reserved Domains

The `system.*` hierarchy is strictly reserved for protocol-level operations.

- `system.auth.grant`: Used to delegate authority to a third-party application (Grantee).
- `system.auth.revoke`: Used to invalidate a previous grant.

4.2. Delegation Mechanism

Authority is a temporary capability. To delegate, a user issues a `system.auth.grant` event where the `subject` is the Grantee's public key and the `content` specifies the constraints (allowed domains and `expires_at`). Example:

```
{
  "types": ["social.forum.post", "social.forum.reply"],
  "expires_at": 1710400000
}
```

The preceding JSON must be string-encoded and escaped for the `content` field. For delegated actions, the application signs with its own key and provides the Grant ID in `auth_id`. Observers MUST confirm the event type is authorized by the Grant and that the event `pubkey` corresponds to the `subject` specified in the `auth_id` event.

5. Revocation

A revocation event invalidates a previously issued `auth_id`, allowing the granter to terminate delegated permissions. This mechanism allows a Granter to unilaterally and definitively terminate delegated permissions, ensuring that the trust graph remains dynamic and that any subsequent actions by the Grantee are immediately recognized as unauthorized by the network.

```
{
  "id": "string (hex)",
  "pubkey": "string (hex)",    // Public key of the Granter
  "created_at": 1710323400,    // Unix timestamp (seconds)
  "type": "system.auth.revoke",
  "subject": "string",         // Public key of the Grantee being revoked
  "auth_id": "",               // Revocations are direct actions (no delegation)
  "content": "{\"target_id\": \"id_of_the_grant_event\"}",
  "sig": "string (hex)"        // Signature of the 'id' by the Granter
}
```

Observers MUST apply revocation based on their local view of event ordering. Any event using a revoked `auth_id` processed after a verified revocation MUST be discarded to prevent authority leakage. This ensures the trust graph remains a self-correcting structure where invalid interaction paths are pruned from the active state.

6. Graph Topology and Sybil Resistance

The protocol structures trust as a data graph, where events are nodes and delegations (`auth_id`) form dependency edges. Resilience against Sybil attacks [5] or collusion does not rely on external oracles, but on the topological analysis of the graph itself. Clusters of synthetic identities created to artificially inflate reputation can be identified and potentially isolated by observers, as they lack historical anchorage (relational weight) with high-density identities in the network.

This topological constraint imposes a structural and economic cost on "identity churn" (the practice of abandoning keys to erase negative history). Since a new identity starts with zero connectivity, reconstructing the relational weight lost by discarding a compromised key requires a significant reinvestment of time and verified interactions. This alignment ensures that maintaining an established identity is computationally and operationally more efficient than rebuilding one, effectively securing the network through incentive compatibility rather than centralized enforcement.

7. Related Work

Decentralized trust is addressed through various architectural lenses. This model builds upon the event-based communication patterns of Nostr [1], incorporating a native delegation hierarchy. Unlike the formal standards for DIDs [6] and Verifiable Credentials [7], which focus on static attestations, this protocol emphasizes dynamic interaction history. Aggregation frameworks like Orange Protocol [8] utilize external metrics for reputation scoring; in contrast, this implementation derives trust from internal graph topology, maintaining alignment with established interactive system principles [9].

8. References

[1] **fiatjaf.** "NIP-01: Basic protocol flow description", 2020.

[2] **Bernstein, D. J.**, **Duif, N.**, **Lange, T.**, **Schwabe, P.**, **Yang, B.** "High-Speed High-Security Signatures". Journal of Cryptographic Engineering, 2012.

[3] **Back, A.** "Hashcash - A Denial of Service Counter-Measure", 2002.

[4] **Nakamoto, S.** "Bitcoin: A Peer-to-Peer Electronic Cash System", 2008.

[5] **Douceur, J. R.** "The Sybil Attack". Proceedings of the First International Workshop on Peer-to-Peer Systems, 2002.

[6] **W3C.** "Decentralized Identifiers (DIDs) v1.0", 2022.

[7] **W3C.** "Verifiable Credentials Data Model v1.1", 2021.

[8] **Orange Protocol.** “Orange: What the Web3 Reputation Protocol is Made of”, Medium, 2021.

[9] **Goldwasser, S., Micali, S., Rackoff, C.** "The Knowledge Complexity of Interactive Proof Systems", 1989.